

Syksy 2025

Luento 8

18.11.2025

- ▶ Kurssipalaute
 - ▶ Kurssilla lopussa kerättävän palautteen lisäksi ns. jatkuva palaute <https://norppa.helsinki.fi>

- ▶ BK107
 - ▶ ma 14.30-16.30
 - ▶ ti 12-16
 - ▶ to 12-16
 - ▶ pe 12-14

- ▶ Käynnissä!
- ▶ Tämä ja pari seuraavaa viikkoa: sprinttien katselmus ja suunnittelu samassa aikaikkunassa
- ▶ Loppudemot
 - ▶ **ke 10.12. klo 10-12 B123**
 - ▶ **to 11.12. klo 10-12 A111**

- ▶ Käytettävissä HY:n GPT-chat
 - ▶ <https://curre.helsinki.fi/chat>

- ▶ Käytettävissä HY:n GPT-chat
 - ▶ <https://curre.helsinki.fi/chat>
- ▶ Keskusteluja ei käytetä koulutusdatana

- ▶ Käytettävissä HY:n GPT-chat
 - ▶ <https://curre.helsinki.fi/chat>
- ▶ Keskusteluja ei käytetä koulutusdatana
- ▶ Kurssimateriaalichat
 - ▶ Ei (ehkä) hallusinoi
 - ▶ Osin herkkä kysymysten sanamuodon suhteen

Ohjelmiston elinkaaren vaiheet

- ▶ Riippumatta tyylistä ja tavasta jolla ohjelmisto tehdään, ohjelmistojen tekemiseen kuuluu
 - ▶ vaatimusten analysointi ja määrittely
 - ▶ **suunnittelu**
 - ▶ **toteutus**
 - ▶ testaus/laadunhallinta
 - ▶ ohjelmiston ylläpito

Ohjelmiston elinkaaren vaiheet

- ▶ Riippumatta tyylistä ja tavasta jolla ohjelmisto tehdään, ohjelmistojen tekemiseen kuuluu
 - ▶ vaatimusten analysointi ja määrittely
 - ▶ **suunnittelu**
 - ▶ **toteutus**
 - ▶ testaus/laadunhallinta
 - ▶ ohjelmiston ylläpito
- ▶ Jakautuu kahteen vaiheeseen:
 - ▶ arkkitehtuurisuunnittelu
 - ▶ olio/komponenttisuunnittelu

Ohjelmiston elinkaaren vaiheet

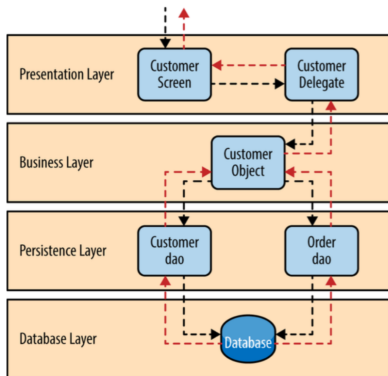
- ▶ Riippumatta tyylistä ja tavasta jolla ohjelmisto tehdään, ohjelmistojen tekemiseen kuuluu
 - ▶ vaatimusten analysointi ja määrittely
 - ▶ **suunnittelu**
 - ▶ **toteutus**
 - ▶ testaus/laadunhallinta
 - ▶ ohjelmiston ylläpito
- ▶ Jakautuu kahteen vaiheeseen:
 - ▶ arkkitehtuurisuunnittelu
 - ▶ olio/komponenttisuunnittelu
- ▶ Näiden lisäksi UI/UX-suunnittelu

Arkkitehtuurityyli

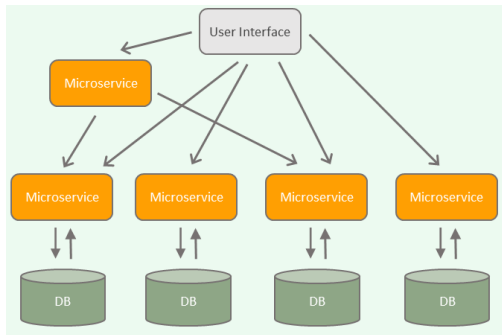
- ▶ Ohjelmiston arkkitehtuuri perustuu yleensä yhteen tai useampaan **arkkitehtuurityyliin** (architectural style)
 - ▶ hyväksi havaittua tapaa strukturoida tietäntyyppisiä sovelluksia
- ▶ Tyylejä suuri määrä
 - ▶ Kerrosarkkitehtuuri
 - ▶ Mikropalveluarkkitehtuuri
 - ▶ MVC
 - ▶ Pipes-and-filters
 - ▶ Repository
 - ▶ Client-server
 - ▶ Publish-subscribe
 - ▶ Event driven
 - ▶ REST
 - ▶ ...

Kerrosarkkitehtuuri

- *Kerros* on kokoelma toisiinsa liittyviä olioita, jotka muodostavat toiminnallisuuden suhteen loogisen kokonaisuuden



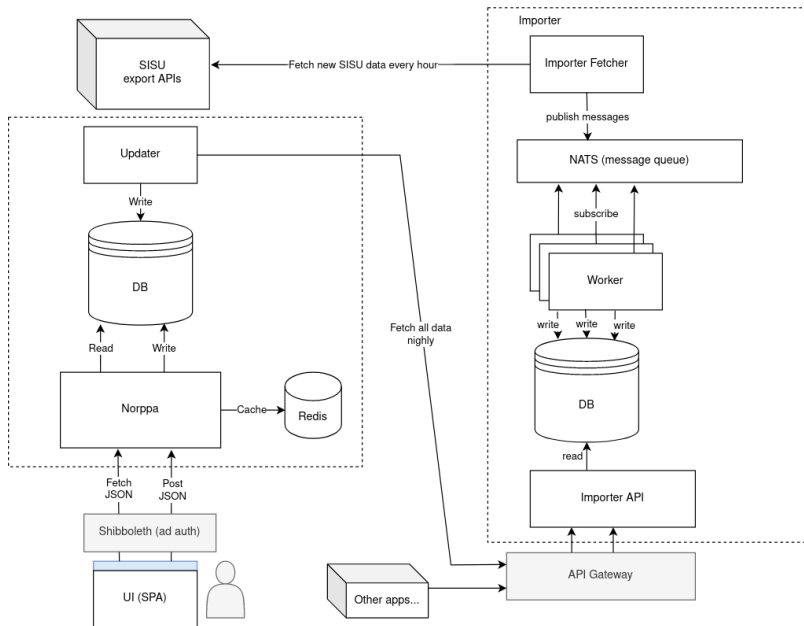
Mikropalveluarkkitehtuuri



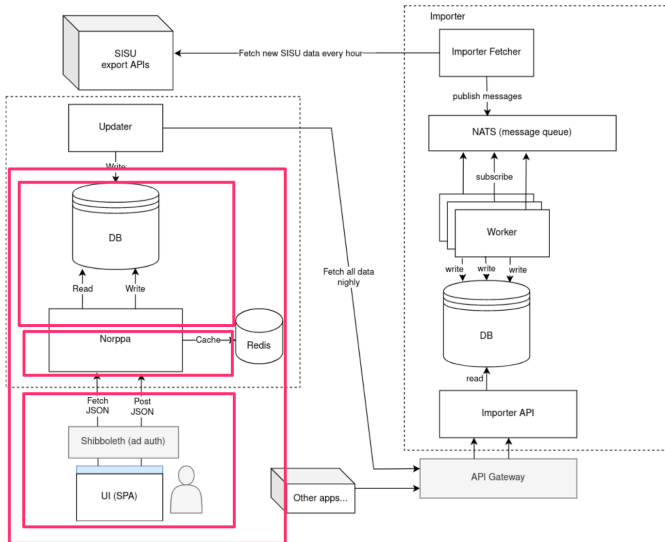
- sovellus koostetaan useista (jopa sadoista) pienistä verkossa toimivista autonomisista palveluista

Kurssipalautejärjestelmä Norppa

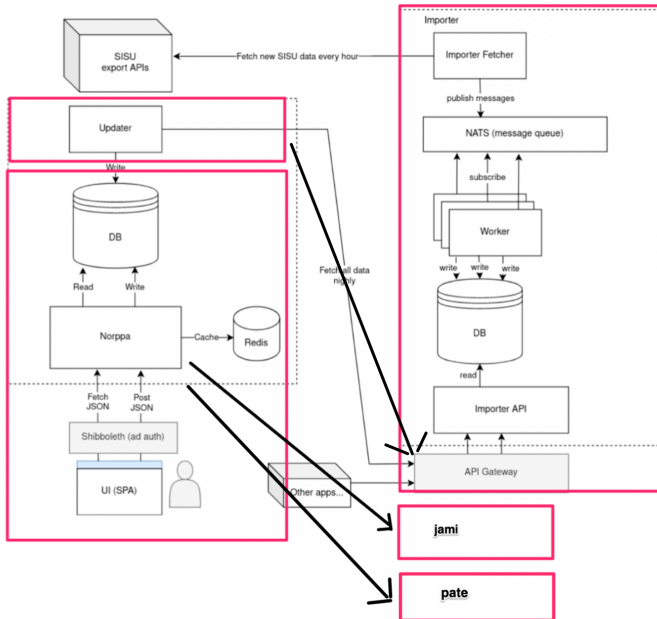
- ▶ Kerrosarkkitehtuuri
- ▶ Mikropalvelu
- ▶ Publish subscribe / Event driven



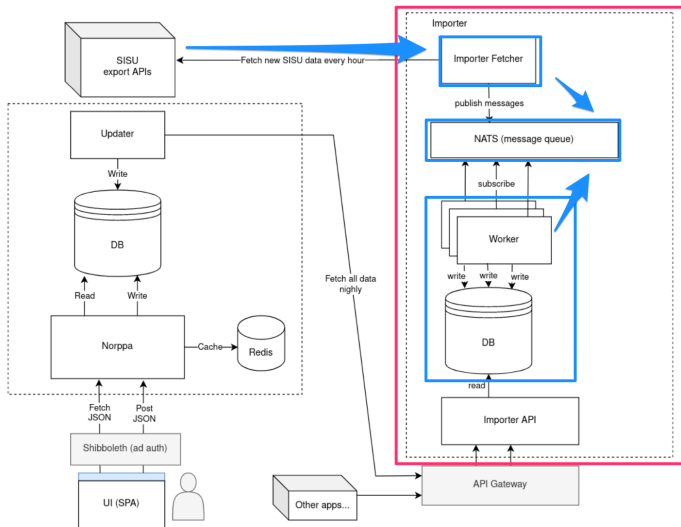
kerrosarkkitehtuuri



mikropalvelut



event driven / messaging



Arkkitehtuurin kuvaamisesta

- ▶ On tilanteita, missä sovelluksen arkkitehtuuri on dokumentoitava jollain tavalla

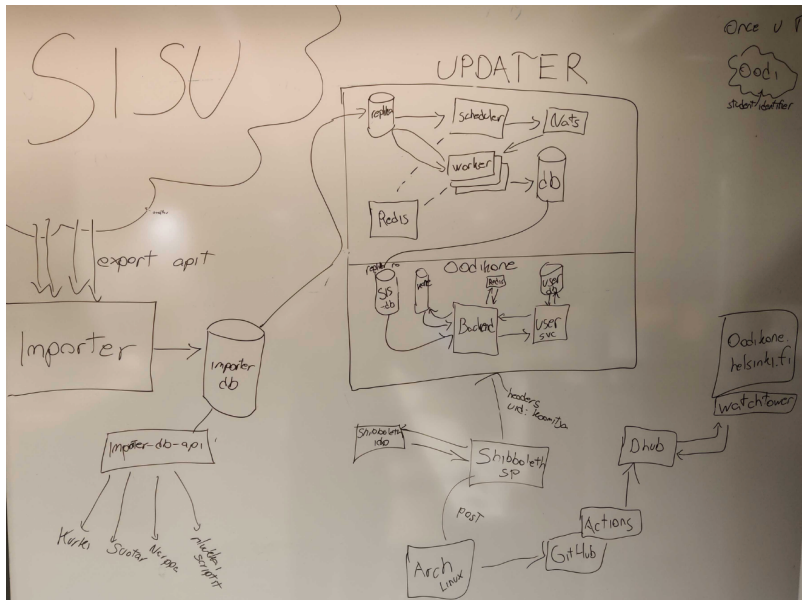
Arkkitehtuurin kuvaamisesta

- ▶ On tilanteita, missä sovelluksen arkkitehtuuri on dokumentoitava jollain tavalla
- ▶ Arkkitehtuurien kuvaamiselle ei ole massa vakiintunutta formaattia
 - ▶ UML:n luokka- ja pakkauskaaviot sekä komponentti- ja sijoittelukaaviot joskus käyttökelpoisia
 - ▶ Useimmiten käytetään epäformaaleja laatikko/nuoli-kaavioita

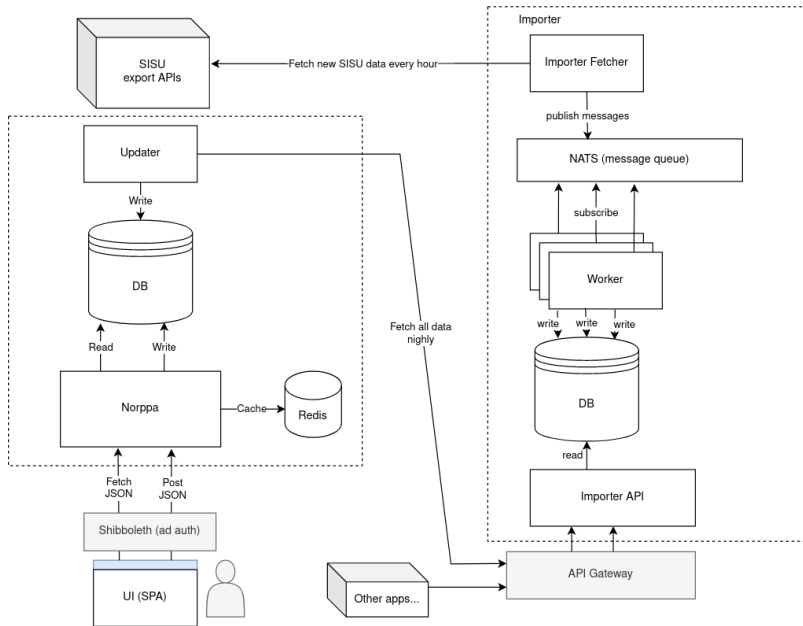
Arkkitehtuurin kuvaamisesta

- ▶ On tilanteita, missä sovelluksen arkkitehtuuri on dokumentoitava jollain tavalla
- ▶ Arkkitehtuurien kuvaamiselle ei ole massa vakiintunutta formaattia
 - ▶ UML:n luokka- ja pakkauskaaviot sekä komponentti- ja sijoittelukaaviot joskus käyttökelpoisia
 - ▶ Useimmiten käytetään epäformaaleja laatikko/nuoli-kaavioita
- ▶ Hyödyllinen arkkitehtuurikuvaus dokumentoi ja perustelee tehtyjä *arkkitehtuurisia valintoja*

Laatikko ja nuoli -kaavio



Hienompi laatikko ja nuoli -kaavio



Arkkitehtuuri ketterissä menetelmissä

Arkkitehtuuri ketterissä menetelmissä

- ▶ Ketterien menetelmien kantava teema on toimivan, asiakkaalle arvoa tuottavan ohjelmiston nopea toimittaminen

Arkkitehtuuri ketterissä menetelmissä

- ▶ Ketterien menetelmien kantava teema on toimivan, asiakkaalle arvoa tuottavan ohjelmiston nopea toimittaminen
- ▶ Arkkitehtuuriin suunnittelu ja dokumentointi on perinteisesti pitkäkestoinen, ohjelmoinnin aloittamista edeltävä vaihe

Arkkitehtuuri ketterissä menetelmissä

- ▶ Ketterien menetelmien kantava teema on toimivan, asiakkaalle arvoa tuottavan ohjelmiston nopea toimittaminen
- ▶ Arkkitehtuuriin suunnittelu ja dokumentointi on perinteisesti pitkäkestoinen, ohjelmoinnin aloittamista edeltävä vaihe
- ▶ Ketterät menetelmät ja “arkkitehtuurivetoinen” ohjelmistotuotanto siis jossain määrin ristiriidassa

Arkkitehtuuri ketterissä menetelmissä

- ▶ Ketterien menetelmien kantava teema on toimivan, asiakkaalle arvoa tuottavan ohjelmiston nopea toimittaminen
- ▶ Arkkitehtuuriin suunnittelu ja dokumentointi on perinteisesti pitkäkestoinen, ohjelmoinnin aloittamista edeltävä vaihe
- ▶ Ketterät menetelmät ja “arkkitehtuurivetoinen” ohjelmistotuotanto siis jossain määrin ristiriidassa
- ▶ Ketterien menetelmien yhteydessä puhutaan *inkrementaalisesta suunnittelusta ja arkkitehtuurista*

Arkkitehtuuri ketterissä menetelmissä

- ▶ Ketterien menetelmien kantava teema on toimivan, asiakkaalle arvoa tuottavan ohjelmiston nopea toimittaminen
- ▶ Arkkitehtuuriin suunnittelu ja dokumentointi on perinteisesti pitkäkestoinen, ohjelmoinnin aloittamista edeltävä vaihe
- ▶ Ketterät menetelmät ja “arkkitehtuurivetoinen” ohjelmistotuotanto siis jossain määrin ristiriidassa
- ▶ Ketterien menetelmien yhteydessä puhutaan *inkrementaalisesta suunnittelusta ja arkkitehtuurista*
- ▶ Arkkitehtuuri mietitään riittävällä tasolla projektin alussa
 - ▶ Jotkut projektit alkavat ns. nollasprintillä ja alustava arkkitehtuuri määritellään tällöin

Arkkitehtuuri ketterissä menetelmissä

- ▶ Ketterien menetelmien kantava teema on toimivan, asiakkaalle arvoa tuottavan ohjelmiston nopea toimittaminen
- ▶ Arkkitehtuuriin suunnittelu ja dokumentointi on perinteisesti pitkäkestoinen, ohjelmoinnin aloittamista edeltävä vaihe
- ▶ Ketterät menetelmät ja “arkkitehtuurivetoinen” ohjelmistotuotanto siis jossain määrin ristiriidassa
- ▶ Ketterien menetelmien yhteydessä puhutaan *inkrementaalisesta suunnittelusta ja arkkitehtuurista*
- ▶ Arkkitehtuuri mietitään riittävällä tasolla projektin alussa
 - ▶ Jotkut projektit alkavat ns. nollasprintillä ja alustava arkkitehtuuri määritellään tällöin
- ▶ “lopullinen” arkkitehtuuri muodostuu iteraatio iteraatiolta samalla kun uutta toiminnallisuutta toteutetaan

Inkrementaalinen arkkitehtuuri

- ▶ Esim. kerrosarkkitehtuurin mukaista sovellusta ei rakenneta “kerros kerrallaan”
 - ▶ Jokaisessa iteraatiossa tehdään pieni pala jokaista kerrosta

Inkrementaalinen arkkitehtuuri

- ▶ Esim. kerrosarkkitehtuurin mukaista sovellusta ei rakenneta “kerros kerrallaan”
 - ▶ Jokaisessa iteraatiossa tehdään pieni pala jokaista kerrosta
- ▶ Alussa ns. *walking skeleton*
 - ▶ sisältää tynkäversiot ohjelmiston komponenttirakenteesta

ui

sovelluslogiikka

tallennus

Inkrementaalinen arkkitehtuuri

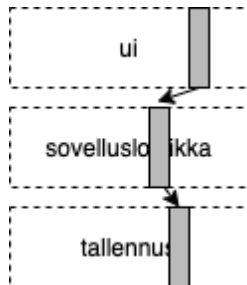
- ▶ Esim. kerrosarkkitehtuurin mukaista sovellusta ei rakenneta “kerros kerrallaan”
 - ▶ Jokaisessa iteraatiossa tehdään pieni pala jokaista kerrosta
- ▶ Alussa ns. *walking skeleton*
 - ▶ sisältää tynkäversiot ohjelmiston komponenttirakenteesta



- ▶ Rakennetaan skeletonin varaan tuotetta story storyltä

Inkrementaalinen arkkitehtuuri

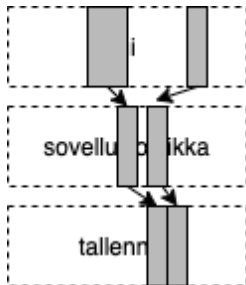
- ▶ Esim. kerrosarkkitehtuurin mukaista sovellusta ei rakenneta “kerros kerrallaan”
 - ▶ Jokaisessa iteraatiossa tehdään pieni pala jokaista kerrosta
- ▶ Alussa ns. *walking skeleton*
 - ▶ sisältää tynkäversiot ohjelmiston komponenttirakenteesta



- ▶ Rakennetaan skeletonin varaan tuotetta story storyltä

Inkrementaalinen arkkitehtuuri

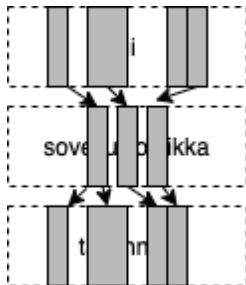
- ▶ Esim. kerrosarkkitehtuurin mukaista sovellusta ei rakenneta “kerros kerrallaan”
 - ▶ Jokaisessa iteraatiossa tehdään pieni pala jokaista kerrosta
- ▶ Alussa ns. *walking skeleton*
 - ▶ sisältää tynkäversiot ohjelmiston komponenttirakenteesta



- ▶ Rakennetaan skeletonin varaan tuotetta story storyltä

Inkrementaalinen arkkitehtuuri

- ▶ Esim. kerrosarkkitehtuurin mukaista sovellusta ei rakenneta “kerros kerrallaan”
 - ▶ Jokaisessa iteraatiossa tehdään pieni pala jokaista kerrosta
- ▶ Alussa ns. *walking skeleton*
 - ▶ sisältää tynkäversiot ohjelmiston komponenttirakenteesta



- ▶ Rakennetaan skeletonin varaan tuotetta story storyltä

- ▶ Perinteisesti arkkitehtuurin luonut *ohjelmistoarkkitehti*
 - ▶ ohjelmoijat velvoitettuja noudattamaan arkkitehtuuria

Arkkitehtuuri ketterissä menetelmissä

- ▶ Perinteisesti arkkitehtuurin luonut *ohjelmistoarkkitehti*
 - ▶ ohjelmoijat velvoitettuja noudattamaan arkkitehtuuria
- ▶ Ketterä idealli: kehitystiimi luo arkkitehtuurin yhdessä
 - ▶ *The best architectures, requirements, and designs emerge from self-organizing teams*

Arkkitehtuuri ketterissä menetelmissä

- ▶ Perinteisesti arkkitehtuurin luonut *ohjelmistoarkkitehti*
 - ▶ ohjelmoijat velvoitettuja noudattamaan arkkitehtuuria
- ▶ Ketterä idealli: kehitystiimi luo arkkitehtuurin yhdessä
 - ▶ *The best architectures, requirements, and designs emerge from self-organizing teams*
- ▶ Etuja:
 - ▶ kehittäjät sitoutuvat paremmin arkkitehtuurin noudattamiseen kuin “norsunluutornissa” olevan arkkitehdin määrittelemään
 - ▶ dokumentaatio voi olla kevyt, tiimi tuntee arkkitehtuurin hengen ja pystyy sitä noudattamaan

Inkrementaalinen arkkitehtuuri: edut ja riskit

- ▶ Oletus: optimaalista arkkitehtuuria ei pystytäkään suunnittelemaan projektin alussa
 - ▶ Jo tehtyjä arkkitehtuuriratkaisuja muutetaan tarvittaessa

Inkrementaalinen arkkitehtuuri: edut ja riskit

- ▶ Oletus: optimaalista arkkitehtuuria ei pystytä suunnittelemaan projektin alussa
 - ▶ Jo tehtyjä arkkitehtuuriratkaisuja muutetaan tarvittaessa
- ▶ Arkkitehtuurin suunnittelussa ketterä pyrkii *välttämään liian aikaisin tehtävää, ehkä turhaksi osoittautuvaa työtä*

Inkrementaalinen arkkitehtuuri: edut ja riskit

- ▶ Oletus: optimaalista arkkitehtuuria ei pystytä suunnittelemaan projektin alussa
 - ▶ Jo tehtyjä arkkitehtuuriratkaisuja muutetaan tarvittaessa
- ▶ Arkkitehtuurin suunnittelussa ketterä pyrkii *välttämään liian aikaisin tehtävää, ehkä turhaksi osoittautuvaa työtä*
- ▶ Inkrementaalinen arkkitehtuuri edellyttää koodilta hyvää sisäistä laatua ja kehittäjiltä kurinalaisuutta
 - ▶ muuten seurauksena on kaaos

TAUKO 10 min

- Sovelluksen arkkitehtuuri antaa raamit, jotka ohjaavat sovelluksen tarkempaa suunnittelua ja toteuttamista

Olio/komponenttisuunnittelu

- ▶ Sovelluksen arkkitehtuuri antaa raamit, jotka ohjaavat sovelluksen tarkempaa suunnittelua ja toteuttamista
- ▶ *Olio- tai komponenttisuunnittelu*
 - ▶ tarkoittaa arkkitehtuuristen komponenttien väliset rajapinnat sekä hahmottelee ohjelman luokka- tai moduulirakenteen

Olio/komponenttisuunnittelu

- ▶ Sovelluksen arkkitehtuuri antaa raamit, jotka ohjaavat sovelluksen tarkempaa suunnittelua ja toteuttamista
- ▶ *Olio- tai komponenttisuunnittelu*
 - ▶ tarkoittaa arkkitehtuuristen komponenttien väliset rajapinnat sekä hahmottelee ohjelman luokka- tai moduulirakenteen
- ▶ Vesiputousmallissa komponenttisuunnittelu tehty ennen ohjelmointia ja dokumentoitu tarkkaan esim. UML:lä

Olio/komponenttisuunnittelu

- ▶ Sovelluksen arkkitehtuuri antaa raamit, jotka ohjaavat sovelluksen tarkempaa suunnittelua ja toteuttamista
- ▶ *Olio- tai komponenttisuunnittelu*
 - ▶ tarkoittaa arkkitehtuuristen komponenttien väliset rajapinnat sekä hahmottelee ohjelman luokka- tai moduulirakenteen
- ▶ Vesiputousmallissa komponenttisuunnittelu tehty ennen ohjelmointia ja dokumentoitu tarkkaan esim. UML:lä
- ▶ Ketterässä tarkka suunnittelu tehdään vasta ohjelmoitaessa

Olio/komponenttisuunnittelu

- ▶ Sovelluksen arkkitehtuuri antaa raamit, jotka ohjaavat sovelluksen tarkempaa suunnittelua ja toteuttamista
- ▶ *Olio- tai komponenttisuunnittelu*
 - ▶ tarkoittaa arkkitehtuuristen komponenttien väliset rajapinnat sekä hahmottelee ohjelman luokka- tai moduulirakenteen
- ▶ Vesiputousmallissa komponenttisuunnittelu tehty ennen ohjelmointia ja dokumentoitu tarkkaan esim. UML:lä
- ▶ Ketterässä tarkka suunnittelu tehdään vasta ohjelmoitaessa
- ▶ Suunnittelussa pyritään maksimoimaan *koodin sisäinen laatu*
 - ▶ helppo ylläpidettävyys ja laajennettavuus

Olio/komponenttisuunnittelu

- ▶ Sovelluksen arkkitehtuuri antaa raamit, jotka ohjaavat sovelluksen tarkempaa suunnittelua ja toteuttamista
- ▶ *Olio- tai komponenttisuunnittelu*
 - ▶ tarkoittaa arkkitehtuuristen komponenttien väliset rajapinnat sekä hahmottelee ohjelman luokka- tai moduulirakenteen
- ▶ Vesiputousmallissa komponenttisuunnittelu tehty ennen ohjelmointia ja dokumentoitu tarkkaan esim. UML:lä
- ▶ Ketterässä tarkka suunnittelu tehdään vasta ohjelmoitaessa
- ▶ Suunnittelussa pyritään maksimoimaan *koodin sisäinen laatu*
 - ▶ helppo ylläpidettävyys ja laajennettavuus
- ▶ Ohjelmistosuunnittelu on “enemmän taidetta kuin tiedettä”, kokemus ja hyvien käytänteiden tuntemus auttaa
 - ▶ kehitetty monia suunnittelumenetelmiä, mikään niistä ei ole vakiintunut

- ▶ Tavoitteena siis **sisäiseltä laadultaan** hyvä koodi

- ▶ Tavoitteena siis **sisäiseltä laadultaan** hyvä koodi
- ▶ *Sisäinen laatu* (internal quality)
 - ▶ onko virheiden jäljitys ja korjaaminen helppoa
 - ▶ onko koodia helppo laajentaa ja jatkokehittää
 - ▶ pystytäänkö koodin toiminnallisuuden oikeellisuus varmistamaan muutoksia tehtäessä

- ▶ Tavoitteena siis **sisäiseltä laadultaan** hyvä koodi
- ▶ *Sisäinen laatu* (internal quality)
 - ▶ onko virheiden jäljitys ja korjaaminen helppoa
 - ▶ onko koodia helppo laajentaa ja jatkokehittää
 - ▶ pystytäänkö koodin toiminnallisuuden oikeellisuus varmistamaan muutoksia tehtäessä
- ▶ Jos sisäinen laatu rapistuu
 - ▶ alkaa vaikuttamaan myös ulkoiseen eli käyttäjän kokemaan laatuun
 - ▶ kehitystiimin velositeetti alkaa tippua

Laadukkaan koodin tuntomerkkejä

Laadukkaan koodin tuntomerkkejä

- ▶ Laadukkaalla koodilla joukko yhteneviä ominaisuuksia, tai *laatuattribuutteja*, esim. seuraavat:
 - ▶ kapselointi
 - ▶ korkea koheesion aste
 - ▶ riippuvuuksien vähäisyys
 - ▶ toisteettomuus
 - ▶ testattavuus
 - ▶ selkeys

Laadukkaan koodin tuntomerkkejä

- ▶ Laadukkaalla koodilla joukko yhteneviä ominaisuuksia, tai *laatuattribuutteja*, esim. seuraavat:
 - ▶ kapselointi
 - ▶ korkea koheesion aste
 - ▶ riippuvuuksien vähäisyys
 - ▶ toisteettomuus
 - ▶ testattavuus
 - ▶ selkeys
- ▶ *Suunnittelumallit* auttavat luomaan sisäisesti laadukasta koodia
 - ▶ kurssin aikana nähty jo *dependency injection*, *repository*
 - ▶ lisää kurssimateriaalissa ja laskareissa

Koodin laatuattribuutti: kapselointi

Koodin laatuattribuutti: kapselointi

- ▶ *Kapselointi ohjelmoinnin peruskursseilla:*
 - ▶ *oliomuuttujat tulee määritellä piilotetuksi ja niille tulee tehdä tarvittaessa aksessorimetodit*

Koodin laatuattribuutti: kapselointi

- ▶ *Kapselointi ohjelmoinnin peruskursseilla:*
 - ▶ *oliomuuttujat tulee määritellä piilotetuksi ja niille tulee tehdä tarvittaessa aksessorimetodit*
- ▶ Olion sisäisen tilan lisäksi kapseloinnin kohde voi olla mm. *käytettävän olion tyyppi, käytetty algoritmi, olioiden luomisen tapa, käytettävän komponentin rakenne*

Koodin laatuattribuutti: kapselointi

- ▶ *Kapselointi* ohjelmoinnin peruskursseilla:
 - ▶ *oliomuuttujat tulee määritellä piilotetuksi ja niille tulee tehdä tarvittaessa aksessorimetodit*
- ▶ Olion sisäisen tilan lisäksi kapseloinnin kohde voi olla mm. *käytettävän olion tyyppi, käytetty algoritmi, olioiden luomisen tapa, käytettävän komponentin rakenne*
- ▶ Näkyy myös arkkitehtuurin tasolla
 - ▶ kerrosarkkitehtuuri: ylempi kerros käyttää ainoastaan alemman kerroksen ulospäin tarjoamaa rajapintaa, muu kapseloitu
 - ▶ mikropalvelut: yksittäinen palvelu kapseloi sisäisen logiikan, tiedon säilytystavan ja tarjoaa ainoastaan verkon välityksellä käytettävän rajapinnan

Koodin laatuattribuutti: koheesio

Koodin laatuattribuutti: koheesio

- ▶ *Koheesio:*
 - ▶ kuinka pitkälle metodin, luokan tai komponentin koodi keskittyy tietyn yksittäisen toiminnallisuuden toteuttamiseen
 - ▶ hyvänä pidetään mahdollisimman korkeaa koheesion astetta

Koodin laatuattribuutti: koheesio

- ▶ *Koheesio:*
 - ▶ kuinka pitkälle metodin, luokan tai komponentin koodi keskittyy tietyn yksittäisen toiminnallisuuden toteuttamiseen
 - ▶ hyvänä pidetään mahdollisimman korkeaa koheesion astetta
- ▶ Luokkatason koheesio
 - ▶ luokan *vastuulla* vain yksi asia, tunnetaan myös nimellä *single responsibility principle*

Koodin laatuattribuutti: koheesio

- ▶ *Koheesio:*
 - ▶ kuinka pitkälle metodin, luokan tai komponentin koodi keskittyy tietyn yksittäisen toiminnallisuuden toteuttamiseen
 - ▶ hyvänä pidetään mahdollisimman korkeaa koheesio-astetta
- ▶ Luokkatason koheesio
 - ▶ luokan *vastuulla* vain yksi asia, tunnetaan myös nimellä *single responsibility principle*
- ▶ Arkkitehtuurin tasolla
 - ▶ kerrosarkkitehtuurin kerrokset samalla abstraktiotasolla, esim. käyttöliittymä tai tietokantarajapinta
 - ▶ mikropalvelu toteuttaa tiettyyn liiketoiminnan tason toiminnallisuuden, esim. suosittelualgoritmin tai käyttäjien hallinnan

Koheesio Flask-sovelluksessa

```
@app.route("/create_todo", methods=["POST"])
def todo_creation():
    content = request.form.get("content")

    try:
        if len(content) < 5:
            raise UserInputError("Todo content length must be greater than 4")

        if len(content) > 100:
            raise UserInputError("Todo content length must be smaller than 100")

        sql = text("INSERT INTO todos (content) VALUES (:content)")
        db.session.execute(sql, { "content": content })
        db.session.commit()

        return redirect("/")
    except Exception as error:
        flash(str(error))
        return redirect("/new_todo")
```

Koheesio Flask-sovelluksessa

```
@app.route("/create_todo", methods=["POST"])
def todo_creation():
    content = request.form.get("content")

    try:
        validate_todo(content)
        create_todo(content)
        return redirect("/")
    except Exception as error:
        flash(str(error))
        return redirect("/new_todo")

# util.py

def validate_todo(content):
    if len(content) < 5:
        raise UserInputError("Todo content length must be greater than 4")

    if len(content) > 100:
        raise UserInputError("Todo content length must be smaller than 100")

# todo_repository.py

def create_todo(content):
    sql = text("INSERT INTO todos (content) VALUES (:content)")
    db.session.execute(sql, { "content": content })
    db.session.commit()
```

Koheesio Flask-sovelluksessa

```
@app.route("/create_todo", methods=["POST"])
def todo_creation():
    content = request.form.get("content")

    try:
        validate_todo(content)
        create_todo(content)
        return redirect("/")
    except Exception as error:
        flash(str(error))
        return redirect("/new_todo")

# util.py

def validate_todo(content):
    if len(content) < 5:
        raise UserInputError("Todo content length must be greater than 4")

    if len(content) > 100:
        raise UserInputError("Todo content length must be smaller than 100")

# todo_repository.py

def create_todo(content):
    sql = text("INSERT INTO todos (content) VALUES (:content)")
    db.session.execute(sql, { "content": content })
    db.session.commit()
```

► *delegoidaan osa vastuista eri komponenteille*

Koodin laatuattribuutti: riippuvuuksien vähäisyys

Koodin laatuattribuutti: riippuvuuksien vähäisyys

- ▶ Pyrkimys korkeaan koheesioon johtaa ohjelmiin, joissa suuri määrä olioita/komponentteja

Koodin laatuattribuutti: riippuvuuksien vähäisyys

- ▶ Pyrkimys korkeaan koheesioon johtaa ohjelmiin, joissa suuri määrä olioita/komponentteja
- ▶ Olioiden oltava keskenään vuorovaikutuksessa toteuttaakseen ohjelman toiminnallisuuden: *paljon keskinäisiä riippuvuuksia*

Koodin laatuattribuutti: riippuvuuksien vähäisyys

- ▶ Pyrkimys korkeaan koheesioon johtaa ohjelmiin, joissa suuri määrä olioita/komponentteja
- ▶ Olioiden oltava keskenään vuorovaikutuksessa toteuttaakseen ohjelman toiminnallisuuden: *paljon keskinäisiä riippuvuuksia*
- ▶ *Riippuvuuksien vähäisyyden* periaate
 - ▶ eliminoidaan *tarpeettomat* riippuvuudet
 - ▶ sekä riippuvuudet konkreettisiin asioihin

Koodin laatuattribuutti: riippuvuuksien vähäisyys

- ▶ Pyrkimys korkeaan koheesioon johtaa ohjelmiin, joissa suuri määrä olioita/komponentteja
- ▶ Olioiden oltava keskenään vuorovaikutuksessa toteuttaakseen ohjelman toiminnallisuuden: *paljon keskinäisiä riippuvuuksia*
- ▶ *Riippuvuuksien vähäisyyden* periaate
 - ▶ eliminoidaan *tarpeettomat* riippuvuudet
 - ▶ sekä riippuvuudet konkreettisiin asioihin
- ▶ Hyödynnetään *dependence injection* -suunnittelumallia

Koodin laatuattribuutti: riippuvuuksien vähäisyys

```
def main():  
    stats = StatisticsService()
```

```
class StatisticsService:  
    def __init__(self):  
        reader = PlayerReader()
```

VS

```
def main():  
    reader = PlayerReader("https://studies.cs.helsinki.fi/nhlstats/2021-22/players.txt")  
    stats = StatisticsService(reader)
```

```
class StatisticsService:  
    def __init__(self, reader):  
        self._players = reader.get_players()
```

Koodin laatuattribuutti: toisteettomuus

Koodin laatuattribuutti: toisteettomuus

- ▶ Aloittelevaa ohjelmoijaa pelotellaan toisteisuuden vaaroista uran ensiaskelista alkaen: älä copypastaa koodia!

Koodin laatuattribuutti: toisteettomuus

- ▶ Aloittelevaa ohjelmoijaa pelotellaan toisteisuuden vaaroista uran ensiaskelista alkaen: älä copypastaa koodia!
- ▶ Alan piireissä toisteisuudesta varoittava periaate kulkee nimellä DRY, don't repeat yourself
 - ▶ *every piece of knowledge must have a single, unambiguous, authoritative representation within a system*

Koodin laatuattribuutti: toisteettomuus

- ▶ Aloittelevaa ohjelmoijaa pelotellaan toisteisuuden vaaroista uran ensiaskelista alkaen: älä copypastaa koodia!
- ▶ Alan piireissä toisteisuudesta varoittava periaate kulkee nimellä DRY, don't repeat yourself
 - ▶ *every piece of knowledge must have a single, unambiguous, authoritative representation within a system*
- ▶ Koodin lisäksi periaate ulottuu koskemaan järjestelmän muitakin osia
 - ▶ tietokantaskeemaa, testejä, build-skriptejä

Koodin laatuattribuutti: toisteettomuus

- ▶ Aloittelevaa ohjelmoijaa pelotellaan toisteisuuden vaaroista uran ensiaskelista alkaen: älä cotypastaa koodia!
- ▶ Alan piireissä toisteisuudesta varoitettava periaate kulkee nimellä DRY, don't repeat yourself
 - ▶ *every piece of knowledge must have a single, unambiguous, authoritative representation within a system*
- ▶ Koodin lisäksi periaate ulottuu koskemaan järjestelmän muitakin osia
 - ▶ tietokantaskeemaa, testejä, build-skriptejä
- ▶ Suoraviivainen cotypaste helppo eliminoida metodien avulla
 - ▶ kaikki toisteisuus ei ole yhtä ilmeistä, monissa suunnittelumalleissa kyse hienovaraisempien toisteisuuden muotojen eliminoinnista

Koodin laatuattribuutti: toisteettomuus

- ▶ Aloittelevaa ohjelmoijaa pelotellaan toisteisuuden vaaroista uran ensiaskelista alkaen: älä copypastaa koodia!
- ▶ Alan piireissä toisteisuudesta varoittava periaate kulkee nimellä DRY, don't repeat yourself
 - ▶ *every piece of knowledge must have a single, unambiguous, authoritative representation within a system*
- ▶ Koodin lisäksi periaate ulottuu koskemaan järjestelmän muitakin osia
 - ▶ tietokantaskeemaa, testejä, build-skriptejä
- ▶ Suoraviivainen copypaste helppo eliminoida metodien avulla
 - ▶ kaikki toisteisuus ei ole yhtä ilmeistä, monissa suunnittelumalleissa kyse hienovaraisempien toisteisuuden muotojen eliminoinnista
- ▶ Hyvä vs. paha copypaste
 - ▶ *three strikes and you refactor*

Koodin laatuattribuutti: testattavuus

Koodin laatuattribuutti: testattavuus

- ▶ Laadukas koodi on helppo testata kattavasti yksikkö- ja integraatiotestein
 - ▶ seuraa yleensä siitä, että koodi koostuu löyhästi kytketyistä, selkeään vastuun omaavista komponenteista

Koodin laatuattribuutti: testattavuus

- ▶ Laadukas koodi on helppo testata kattavasti yksikkö- ja integraatiotestein
 - ▶ seuraa yleensä siitä, että koodi koostuu löyhästi kytketyistä, selkeän vastuun omaavista komponenteista
- ▶ Hyvää testattavuutta auttaa turhien riippuvuuksien eliminointi dependency injection -periaatteen avulla

Koodin laatuattribuutti: testattavuus

- ▶ Laadukas koodi on helppo testata kattavasti yksikkö- ja integraatiotestein
 - ▶ seuraa yleensä siitä, että koodi koostuu löyhästi kytketyistä, selkeän vastuun omaavista komponenteista
- ▶ Hyvää testattavuutta auttaa turhien riippuvuuksien eliminointi dependency injection -periaatteen avulla
- ▶ Test driven development tuottaa varmuudella hyvin testattavissa olevaa koodia

Koodin laatuattribuutti: selkeys ja luettavuus

- ▶ Perinteisesti ajateltu että koodi kryptistä ja vaikeasti luettavaa
 - ▶ yleistä C-kielessä, pyritty esim. optimoimaan tehokkuutta ja muistinkäyttöä

Koodin laatuattribuutti: selkeys ja luettavuus

- ▶ Perinteisesti ajateltu että koodi kryptistä ja vaikeasti luettavaa
 - ▶ yleistä C-kielessä, pyritty esim. optimoimaan tehokkuutta ja muistinkäyttöä
- ▶ Nykytrendi: tehdän koodia, joka nimeämisen sekä rakenteen kautta ilmaisee hyvin sen, mitä koodi tekee

Koodin laatuattribuutti: selkeys ja luettavuus

- ▶ Perinteisesti ajateltu että koodi kryptistä ja vaikeasti luettavaa
 - ▶ yleistä C-kielessä, pyritty esim. optimoimaan tehokkuutta ja muistinkäyttöä
- ▶ Nykytrendi: tehdän koodia, joka nimeämisen sekä rakenteen kautta ilmaisee hyvin sen, mitä koodi tekee
- ▶ Miksi selkeä koodi on tärkeää?
 - ▶ joidenkin arvioiden mukaan jopa 90% “ohjelmointiin” kuluvasta ajasta menee olemassa olevan koodin lukemiseen
 - ▶ oma aikoinaan niin selkeä koodi, ei enää olekaan yhtä selkeää parin kuukauden kuluttua

Code smell

- ▶ Koodi ei ole aina hyvää...

Code smell

- ▶ Koodi ei ole aina hyvää...
- ▶ Martin Fowlerin mukaan
 - ▶ *koodihaju* (code smell) on helposti huomattava merkki siitä että koodissa on jotain pielessä
 - ▶ jopa aloitteleva ohjelmoija saattaa pystyä havaitsemaan koodihajun
 - ▶ sen takana oleva todellinen syy voi olla jossain syvemmällä

Code smell

- ▶ Koodi ei ole aina hyvää...
- ▶ Martin Fowlerin mukaan
 - ▶ *koodihaju* (code smell) on helposti huomattava merkki siitä että koodissa on jotain pielessä
 - ▶ jopa aloitteleva ohjelmoija saattaa pystyä havaitsemaan koodihajun
 - ▶ sen takana oleva todellinen syy voi olla jossain syvemmällä
- ▶ Koodihaju siis kertoo, että syystä tai toisesta *koodin sisäinen laatu* ei ole parhaalla mahdollisella tasolla

Koodihajuja

- ▶ Koodihajuja on hyvin monenlaisia ja monentasoisia
- ▶ Esimerkkejä helposti tunnistettavista hajuista:
 - ▶ toisteinen koodi
 - ▶ liian pitkät metodit
 - ▶ luokat joissa on liikaa oliomuuttujia
 - ▶ luokat joissa on liikaa koodia
 - ▶ metodien liian pitkät parametrilistat
 - ▶ epäselkeät muuttujien, metodien tai luokkien nimet
 - ▶ kommentit

Koodihajuja

- ▶ Koodihajuja on hyvin monenlaisia ja monentasoisia
- ▶ Esimerkkejä helposti tunnistettavista hajuista:
 - ▶ toisteinen koodi
 - ▶ liian pitkät metodit
 - ▶ luokat joissa on liikaa oliomuuttujia
 - ▶ luokat joissa on liikaa koodia
 - ▶ metodien liian pitkät parametrilistat
 - ▶ epäselkeät muuttujien, metodien tai luokkien nimet
 - ▶ kommentit
- ▶ Pari monimutkaisempaa
 - ▶ Primitive obsession
 - ▶ Shotgun surgery

Refaktorointi

- ▶ Lääke koodin sisäisen laadun ongelmiin on *refaktorointi*
 - ▶ koodin toiminnallisuuden ennallaan pitävä sisäisen rakenteen muutos

- ▶ Lääke koodin sisäisen laadun ongelmiin on *refaktorointi*
 - ▶ koodin toiminnallisuuden ennallaan pitävä sisäisen rakenteen muutos
- ▶ Koodin rakennetta parantavia refaktorointeja on lukuisia, mm.
 - ▶ *rename variable/method/class*
 - ▶ *extract method*
 - ▶ *move field/method*
 - ▶ *extract superclass*

Refaktorointi

- ▶ Lääke koodin sisäisen laadun ongelmiin on *refaktorointi*
 - ▶ koodin toiminnallisuuden ennallaan pitävä sisäisen rakenteen muutos
- ▶ Koodin rakennetta parantavia refaktorointeja on lukuisia, mm.
 - ▶ *rename variable/method/class*
 - ▶ *extract method*
 - ▶ *move field/method*
 - ▶ *extract superclass*
- ▶ Osa pystytään tekemään sovelluskehitysympäristön avustamana
 - ▶ helpompaa staattisesti tyyplitetyillä kielillä kuten Java

Miten refaktorointi kannattaa tehdä

- ▶ Refaktoroinnin edellytys on kattavien testien olemassaolo

Miten refaktorointi kannattaa tehdä

- ▶ Refaktoroinnin edellytys on kattavien testien olemassaolo
- ▶ Kannattaa ehdottomasti edetä pienin askelin
 - ▶ yksi hallittu muutos kerrallaan
 - ▶ testit suoritettava mahdollisimman usein

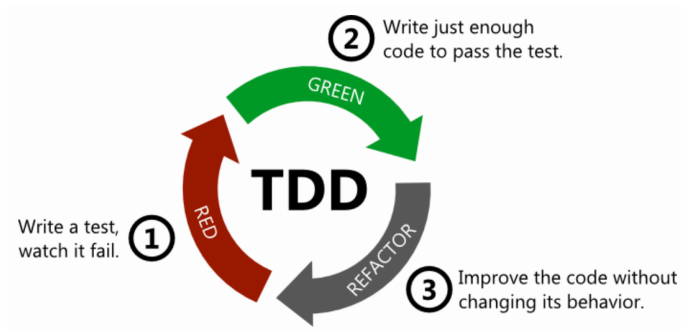
Miten refaktorointi kannattaa tehdä

- ▶ Refaktoroinnin edellytys on kattavien testien olemassaolo
- ▶ Kannattaa ehdottomasti edetä pienin askelin
 - ▶ yksi hallittu muutos kerrallaan
 - ▶ testit suoritettava mahdollisimman usein
- ▶ Refaktorointia kannattaa suorittaa lähes jatkuvasti
 - ▶ pitää koodin rakenteen selkeänä ja helpottaa sekä nopeuttaa koodin laajentamista

Miten refaktorointi kannattaa tehdä

- ▶ Refaktoroinnin edellytys on kattavien testien olemassaolo
- ▶ Kannattaa ehdottomasti edetä pienin askelin
 - ▶ yksi hallittu muutos kerrallaan
 - ▶ testit suoritettava mahdollisimman usein
- ▶ Refaktorointia kannattaa suorittaa lähes jatkuvasti
 - ▶ pitää koodin rakenteen selkeänä ja helpottaa sekä nopeuttaa koodin laajentamista
- ▶ Osa refaktoroinnista on helppoa ja suoraviivaista, aina ei näin ole
 - ▶ joskus tarve tehdä isoja, jopa viikkojen kestoisia refaktorointeja joissa ohjelman rakenne muuttuu paljon

Refaktorointi tärkeä osa Test driven development -menetelmää



1. Kirjoitetaan sen verran testiä että testi ei mene läpi
2. Kirjoitetaan koodia sen verran, että testi menee läpi
3. **Jos huomataan koodin rakenteen menneen huonoksi refaktoroidaan koodin rakenne paremmaksi**
4. Jatketaan askeleesta 1

Avoim yliopisto: Test-Driven Development 4 + 1 cr

← → ↺ tdd.mooc.fi ☆ 📌

Test-Driven Development

Course overview

Practicalities (2024 Spring)

Exercises and schedule

Chapter 1: What is TDD

Chapter 2: Refactoring and design

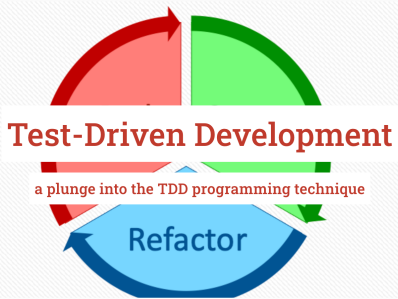
Chapter 3: The Untestables

Chapter 4: Legacy code

Chapter 5: Advanced techniques

Chapter 6: To infinity and beyond

About the material



► Esko Luontola Nitor (Suomen johtava TDD-asiantuntija)

- ▶ Koodi ei ole aina laadultaan optimaalista

Tekninen velka

- ▶ Koodi ei ole aina laadultaan optimaalista
- ▶ Huonoa suunnittelua tai/ja ohjelmointia kuvaa käsite *tekkinen velka* (technical debt)

Tekninen velka

- ▶ Koodi ei ole aina laadultaan optimaalista
- ▶ Huonoa suunnittelua tai/ja ohjelmointia kuvaa käsite *tekninen velka* (technical debt)
- ▶ Piittaamattomalla ja laiskalla ohjelmoinnilla/suunnittelulla saadaan ehkä nopeasti aikaan jotain
 - ▶ hätäinen ratkaisu tullaan maksamaan korkoineen takaisin *jos* ohjelmaa on tarkoitus laajentaa

Tekninen velka

- ▶ Koodi ei ole aina laadultaan optimaalista
- ▶ Huonoa suunnittelua tai/ja ohjelmointia kuvaa käsite *tekkinen velka* (technical debt)
- ▶ Piittaamattomalla ja laiskalla ohjelmoinnilla/suunnittelulla saadaan ehkä nopeasti aikaan jotain
 - ▶ hätäinen ratkaisu tullaan maksamaan korkoineen takaisin *jos* ohjelmaa on tarkoitus laajentaa
- ▶ Jos korkojen maksun aikaa ei koskaan tule, voi “huono koodi” olla asiakkaan etu
 - ▶ esim. minimal viable product (MVP)

Tekninen velka

- ▶ Koodi ei ole aina laadultaan optimaalista
- ▶ Huonoa suunnittelua tai/ja ohjelmointia kuvaa käsite *tekniinen velka* (technical debt)
- ▶ Piittaamattomalla ja laiskalla ohjelmoinnilla/suunnittelulla saadaan ehkä nopeasti aikaan jotain
 - ▶ hätäinen ratkaisu tullaan maksamaan korkoineen takaisin *jos* ohjelmaa on tarkoitus laajentaa
- ▶ Jos korkojen maksun aikaa ei koskaan tule, voi “huono koodi” olla asiakkaan etu
 - ▶ esim. minimal viable product (MVP)
- ▶ Tekninen velka voi olla järkevää tai jopa välttämätöntä
 - ▶ voidaan saada tuote nopeammin markkinoille tekemällä tietoisesti huonoa designia, joka korjataan myöhemmin

- ▶ Kaikki tekninen velka ei samanlaista, taustalla voi olla
 - ▶ holtittomuus, osaamattomuus, tietämättömyys tai tarkoituksella tehty päätös

- ▶ Kaikki tekninen velka ei samanlaista, taustalla voi olla
 - ▶ holtittomuus, osaamattomuus, tietämättömyys tai tarkoituksella tehty päätös
- ▶ Martin Fowler jaottelee teknisen velan neljään eri luokkaan:
 - ▶ Reckless and deliberate: “we do not have time for design”
 - ▶ Reckless and inadvertent: “what is layering”?
 - ▶ Prudent and inadvertent: “now we know how we should have done it”
 - ▶ **Prudent and deliberate: “we must ship now and will deal with consequences”**

- ▶ Kaikki tekninen velka ei samanlaista, taustalla voi olla
 - ▶ holtittomuus, osaamattomuus, tietämättömyys tai tarkoituksella tehty päätös
- ▶ Martin Fowler jaottelee teknisen velan neljään eri luokkaan:
 - ▶ Reckless and deliberate: “we do not have time for design”
 - ▶ Reckless and inadvertent: “what is layering”?
 - ▶ Prudent and inadvertent: “now we know how we should have done it”
 - ▶ **Prudent and deliberate: “we must ship now and will deal with consequences”**
- ▶ Joskus tekninen velka pakottaa koodaamaan koko järjestelmän uudelleen